

Statistical Analysis of Football Matches Data

2024-05-10

Statistical Learning Project aa: 2023/2024

Group members: Flavio Agostini, Niccolò Castellan

1 Defining our goals:

We present a project based on football data. Our main goal will be to answer the question: What are the most important factors that contribute to winning a football match? To answer this question data from more than 300 matches has been gathered, both from high-profile, and average-performing teams. Along the way, we hope to gain even more insights about what it takes to be a winning team.

2 Data collection:

As a first step in data collection, it has been established which parameters could relate to a team's success during a football match. To this end, a questionnaire was submitted to 12 football players with at least 5 years of experience in a semi-pro or pro league. The questionnaire asked them which were the most important factors to win a football match, between:

- Ball possession
- Shots
- Shots on target
- Shots precision (Shots on target / Shots)
- Passage precision
- Km traveled by players
- Quality of the roster (expressed as sum of the values of players)
- Discipline (yellow and red cards)
- Fouls committed and suffered
- Tackles won

From these factors the most important, according to the participants of the interview, were passage precision, shots on target, shots precision, km traveled and tackles won. Participants were free to add suggestions, the most common ones have been: tactics employed, tactical discipline, motivation, referee behavior, field advantage, injuries, level of team-play, and experience of the team/roaster. To decide which variables to include in the final dataset, on top of this survey, considerations about data availability and numerical representation of the features have been made. The features that best suited these criteria and will thus be included in the final dataset are:

Variable	Variable name	Explanation
Field advantage	FieldAdvantage	valued H for “Home”, or A for “Away”
Possession	BallPoss	expressed as a percentage
Attempted passages	PassAtt	
Successful passages	PassSucc	
Passage precision	PassPrec	expressed as a percentage
Shots	Shots	
Shots on target	ShotsOnT	
Shots precision	ShotsPrec	expressed as a percentage
Roster quality	RosterQuality	expressed as sum of market value for each player competing in the match, including substitutions
Knowledge of the league	Knowledge	expressed as average of matches played in the team’s championship by every player competing in the match
Yellow cards	YellowC	
Red cards	RedC	
Fouls committed	FoulsC	
Fouls taken	FoulsT	
Attempted tackles	TacklesAtt	
Successful tackles	TacklesW	
Tackles efficiency	TacklesWRatio	expressed as a percentage
Air duels won	AirDuelW	expressed as a percentage
Attempted dribbles	DribAtt	
Successful dribbles	DribW	
Dribbles efficiency	DribWRation	expressed as a percentage

On top of these features, other four variables, that will serve as target variables, have been gathered: Goals scored, goals taken, outcome of the match, and points gained.

10 teams have been chosen for data collection. Of these, 5 teams are the winners of the most prestigious national leagues in Europe. The other 5 have been randomly picked from the same leagues, between teams that didn’t perform well enough to qualify for the continental cups but avoided placements that led to relegation. “League Winners” group, or “Group 1”:

1. Barclays Premier League (Inghilterra) – Chelsea
2. La Liga (Spagna) – Real Madrid
3. Bundesliga (Germania) – Bayern Munich
4. Ligue 1 (Francia) – AS Monaco
5. Serie A (Italia) – Juventus

“Control” group, or “Group 0”:

1. Barclays Premier League (Inghilterra) – Everton, pos. 7
2. La Liga (Spagna) – Celta Vigo, pos. 13
3. Bundesliga (Germania) – Eintracht Frankfurt, pos. 11
4. Ligue 1 (Francia) – Montpellier, pos. 15
5. Serie A (Italia) – Bologna, pos. 15

Data was acquired during the year 2018, as part of a BSc thesis project in Physical Education. The data has been collected manually from the Internet. To validate it, for each value a cross-check has been performed

between the website whoscored, and the official ESPN website. The only exception to this concerns the parameter “Quality of the roster”. To calculate it market values given by transfermarkt have been used. The data has then been saved in a Microsoft Excel spreadsheet for ease of organization and later converted into CSV format to be compatible with the R software work environment, where the statistical analysis will be performed.

We import our data and assign it to a data frame ‘df’:

```
df <- read.csv('C:/Users/saius/Documents/SL_project/DatasetR_eng_fin.csv', header=TRUE, sep=';',
               fileEncoding="UTF-8")
```

3 Data cleaning, filtering and preprocessing:

Our data has been manually collected and curated so it should already be complete, we nevertheless check for any missing value:

```
anyNA(df)
```

```
## [1] FALSE
```

Using information from “ID” column, we create a new column that identifies as “1” members of the “League winners” group, and as “0” members of the “Control” group:

```
df$Group <- ifelse(grepl("Ca", df$ID), "1",
                  ifelse(grepl("Co", df$ID), "0", NA))
df$Group <- as.integer(df$Group)
```

We inspect the data type of our data frame’s columns:

```
str(df)
```

```
## 'data.frame': 372 obs. of 29 variables:
## $ ID : chr "PLCa1" "PLCa2" "PLCa3" "PLCa4" ...
## $ Match : chr "15/08/2016 Chelsea-West Ham United" "20/08/2016 Watford-Chelsea" "27/08/2016" ...
## $ FieldAdvantage: chr "H" "A" "H" "A" ...
## $ OutcomeWL : chr "W" "W" "W" "D" ...
## $ PointsWon : int 3 3 3 1 0 0 3 3 3 3 ...
## $ Result : chr "2 a 1" "1 a 2" "3 a 0" "2 a 2" ...
## $ GoalsMade : int 2 2 3 2 1 0 2 3 4 2 ...
## $ GoalsTaken : int 1 1 0 2 2 3 0 0 0 0 ...
## $ BallPoss : chr "61,70%" "62,40%" "60,10%" "55,10%" ...
## $ Shots : int 16 13 22 28 12 9 22 16 14 13 ...
## $ ShotsOnT : int 7 4 10 7 4 2 9 6 6 7 ...
## $ ShotsPrec : chr "43,75%" "30,77%" "45,46%" "25,00%" ...
## $ PassAtt : int 480 488 545 416 474 445 460 458 364 325 ...
## $ PassSucc : int 556 560 626 488 540 535 554 553 447 409 ...
## $ PassPrec : chr "86,33%" "87,14%" "87,06%" "85,25%" ...
## $ RosterQuality : int 365500000 378500000 365500000 387500000 422500000 383000000 378000000 317000000 ...
## $ Knowledge : chr "136,3571429" "149,7142857" "136,3571429" "154,5" ...
## $ YellowC : int 5 3 2 4 1 2 2 1 3 0 ...
## $ RedC : int 0 0 0 0 0 0 0 0 0 0 ...
```

```
## $ FoulsC      : int  16 10 6 9 6 11 15 6 7 16 ...
## $ FoulsT      : int  16 20 9 17 13 9 13 12 17 13 ...
## $ TacklesW     : int  18 17 9 18 18 24 19 19 18 27 ...
## $ TacklesAtt   : int  21 26 16 23 34 35 22 24 25 35 ...
## $ TacklesWRatio : chr  "85,71%" "65,39%" "56,25%" "78,26%" ...
## $ AirDuelW     : chr  "49%" "57%" "50%" "53%" ...
## $ DribW        : int  16 11 11 9 22 9 18 17 12 13 ...
## $ DribAtt      : int  33 25 23 22 35 20 30 23 21 21 ...
## $ DribWRatio   : chr  "48,49%" "44,00%" "47,83%" "40,91%" ...
## $ Group        : int  1 1 1 1 1 1 1 1 1 1 ...
```

Percentages variables are stored as characters. we would like them to be represented as numerical:

```
df$BallPoss <- as.numeric(sub(",", ".", sub("%", "", df$BallPoss)))
df$ShotsPrec <- as.numeric(sub(",", ".", sub("%", "", df$ShotsPrec)))
df$PassPrec <- as.numeric(sub(",", ".", sub("%", "", df$PassPrec)))
df$TacklesWRatio <- as.numeric(sub(",", ".", sub("%", "", df$TacklesWRatio)))
df$AirDuelW <- as.numeric(sub(",", ".", sub("%", "", df$AirDuelW)))
df$DribWRatio <- as.numeric(sub(",", ".", sub("%", "", df$DribWRatio)))
```

The Knowledge column is a character as well, we change this to numeric as well:

```
df$Knowledge <- as.numeric(sub(",", ".", df$Knowledge))
```

Columns ID and match are not useful for our model and the column “result” column is redundant with goals made and taken, we remove them.

```
df$ID <- NULL
df$Match <- NULL
df$Result <- NULL
```

To handle categorical variables, we proceed with label encoding. We want in the FieldAdvantage column, to show Home matches “H” as “1” and Away matches “A” as “0”.

```
df$FieldAdvantage <- ifelse(df$FieldAdvantage == "H", 1,
                             ifelse(df$FieldAdvantage == "A", 0, NA))
```

We also want losses “L” as “-1”, wins “W” as “1” and draws “D” as “0” in the OutcomeWL column. This column can be redundant with respect to Points won. For the moment, we will keep both.

```
df$OutcomeWL <- ifelse(df$OutcomeWL == "W", 1,
                        ifelse(df$OutcomeWL == "D", 0,
                               ifelse(df$OutcomeWL == "L", -1, NA)))
```

We confirm that every column is now numerical or integer.

```
str(df)
```

```
## 'data.frame': 372 obs. of 26 variables:
## $ FieldAdvantage: num 1 0 1 0 1 0 0 1 1 0 ...
## $ OutcomeWL : num 1 1 1 0 -1 -1 1 1 1 1 ...
```

```
## $ PointsWon      : int  3 3 3 1 0 0 3 3 3 3 ...
## $ GoalsMade      : int  2 2 3 2 1 0 2 3 4 2 ...
## $ GoalsTaken     : int  1 1 0 2 2 3 0 0 0 0 ...
## $ BallPoss       : num  61.7 62.4 60.1 55.1 52.7 50.2 59.1 54.9 43.9 45.2 ...
## $ Shots          : int  16 13 22 28 12 9 22 16 14 13 ...
## $ ShotsOnT       : int  7 4 10 7 4 2 9 6 6 7 ...
## $ ShotsPrec      : num  43.8 30.8 45.5 25 33.3 ...
## $ PassAtt        : int  480 488 545 416 474 445 460 458 364 325 ...
## $ PassSucc       : int  556 560 626 488 540 535 554 553 447 409 ...
## $ PassPrec       : num  86.3 87.1 87.1 85.2 87.8 ...
## $ RosterQuality  : int  365500000 378500000 365500000 387500000 422500000 383000000 378000000 317000000
## $ Knowledge      : num  136 150 136 154 128 ...
## $ YellowC        : int  5 3 2 4 1 2 2 1 3 0 ...
## $ RedC           : int  0 0 0 0 0 0 0 0 0 0 ...
## $ FoulsC         : int  16 10 6 9 6 11 15 6 7 16 ...
## $ FoulsT         : int  16 20 9 17 13 9 13 12 17 13 ...
## $ TacklesW       : int  18 17 9 18 18 24 19 19 18 27 ...
## $ TacklesAtt     : int  21 26 16 23 34 35 22 24 25 35 ...
## $ TacklesWRatio  : num  85.7 65.4 56.2 78.3 52.9 ...
## $ AirDuelW       : num  49 57 50 53 48 61 60 58 44 47 ...
## $ DribW          : int  16 11 11 9 22 9 18 17 12 13 ...
## $ DribAtt        : int  33 25 23 22 35 20 30 23 21 21 ...
## $ DribWRatio     : num  48.5 44 47.8 40.9 62.9 ...
## $ Group          : int  1 1 1 1 1 1 1 1 1 1 ...
```

Since for our main goal, the target variable is OutcomeWL, we check if it is balanced:

```
prop.table(table(df$OutcomeWL))
```

```
##
##      -1      0      1
## 0.2688172 0.1801075 0.5510753
```

The has a majority of winning matches (55%), and the remaining 45% split between losses and draws. The question that we want to answer by building a model with this target variable is: - Which are the most important factors that contribute to a win? We therefore decided to just concern ourselves with won matches and non-won matches. We achieve this by joining draws and losses in a single category.

```
df$OutcomeWL[df$OutcomeWL == -1] <- 0
prop.table(table(df$OutcomeWL))
```

```
##
##      0      1
## 0.4489247 0.5510753
```

We now have “0” for losses and draws and “1” for wins. To assess the quality of our data and visualize the distribution of the various features we use Q-Q plots. These plots are rendered for each non-target variable, except “FieldAdvantage”, which contains 50% zeroes and 50% ones.

```
par(mar=c(1,1,1,1))
par(mfrow=c(5, 4))
columns <- c("BallPoss", "Shots", "ShotsOnT", "ShotsPrec", "PassAtt", "PassSucc", "PassPrec",
```

```

"RosterQuality", "Knowledge", "YellowC", "RedC", "FoulsC", "FoulsT", "TacklesW", "TacklesA",
"TacklesWRatio", "AirDuelW", "DribW", "DribAtt", "DribWRatio")

```

```

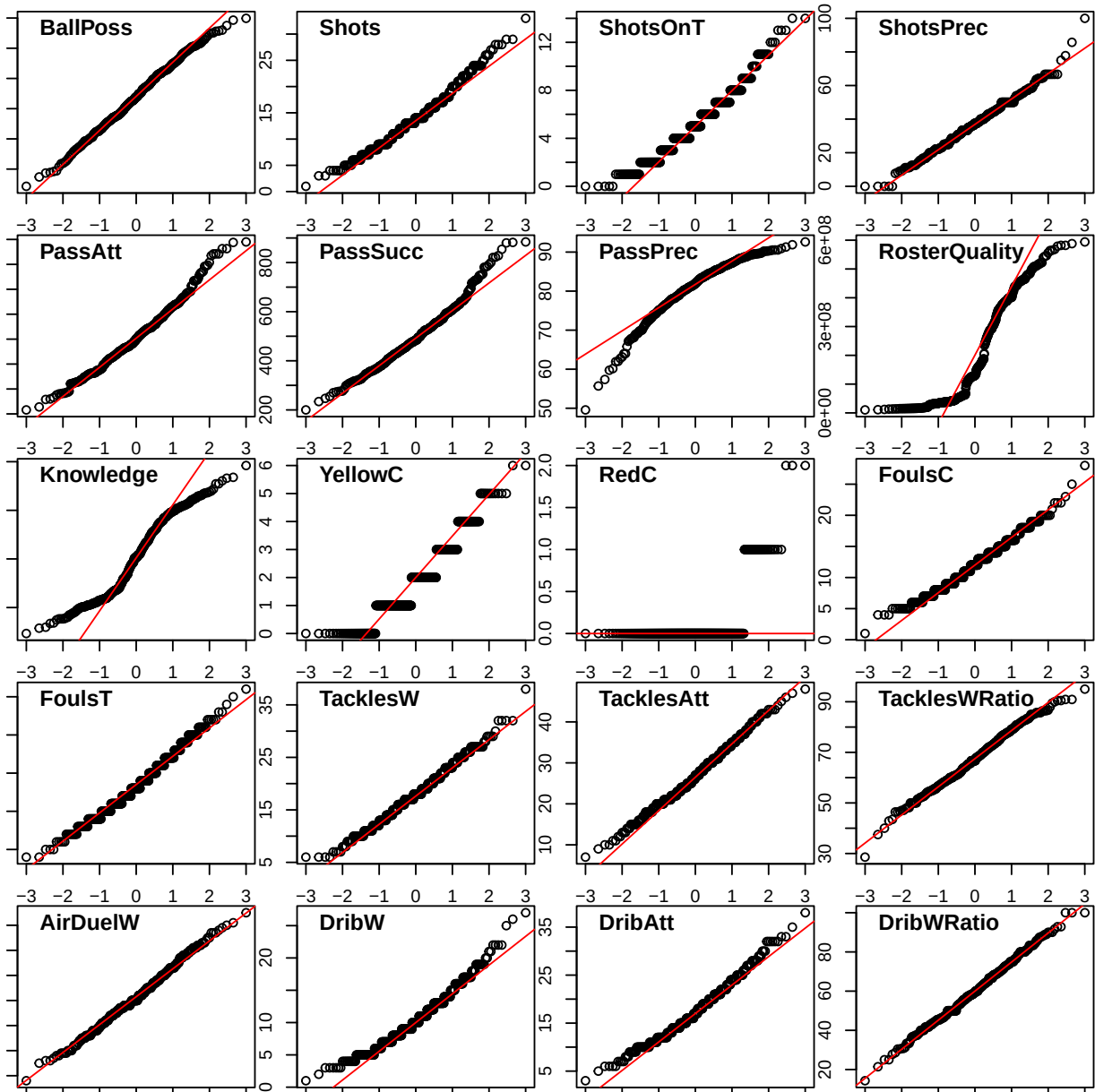
# Loop through each column name and create the QQ plot

```

```

for (col in columns) {
  qqnorm(df[[col]], main='')
  qqline(df[[col]], col = "red")
  title <- paste(col)
  text(x = par("usr")[1] + 0.05 * diff(par("usr")[1:2]),
       y = par("usr")[4] - 0.1 * diff(par("usr")[3:4]),
       labels = title, pos = 4, cex = 1.2, font = 2)
}

```



We notice that most features tend to have a close to normal behavior, even if some of them (Shots, ShotsOnT, PassAtt, PassSucc, DribW, and DribAtt) present bigger tails if compared with normal distribution, and PassPrec, on the contrary, shows very thin tails. The two variables that depart for normality the most seem to be “RoasterQuality” and “Knowledge” which present an “S” shape on the Q-Q plot. This indicates that the data for these variables is strongly skewed on the left, with many low-valued examples and very few high-valued ones. We decided to address this problem with binning.

```
# function to visualize the range of values for each bin
clean_bin <- function(bin) {
  numbers <- gsub("[^0-9e+.-]", "", unlist(strsplit(bin, ",")))
  formatted_numbers <- format(as.numeric(numbers), big.mark = ",", scientific = FALSE)
  cleaned_bin <- paste0("(", formatted_numbers[1], ", ", formatted_numbers[2], ")")
  return(cleaned_bin)
}
```

A new data frame that will contain normalized values is created. We start with RoasterQuality binning.

```
df_norm <- df

# binning of roster quality feature
df_norm$RosterQuality <- cut(df$RosterQuality, breaks = 10)
levels.roster <- levels(df_norm$RosterQuality)

# Apply the cleaning function to all bins
levels.roster <- sapply(levels.roster, clean_bin)

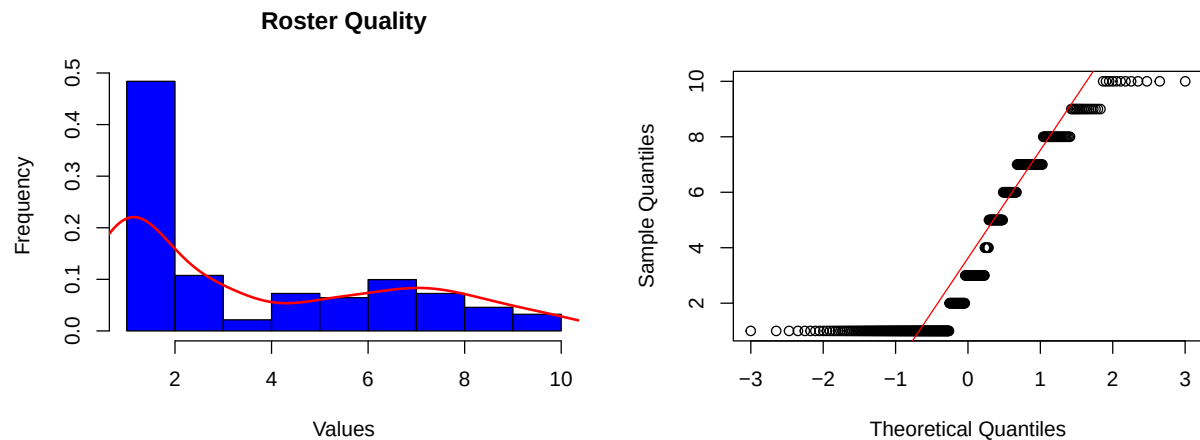
# make it numeric
df_norm$RosterQuality <- as.numeric(df_norm$RosterQuality)
```

And visualize the result:

```
par(mfrow=c(1, 2))
hist(df_norm$RosterQuality, main = "Roster Quality", xlab = "Values", ylab = "Frequency",
     col = "blue", border = "black", probability = TRUE)

lines(density(df_norm$RosterQuality), col = "red", lwd = 2)

qqnorm(df_norm$RosterQuality, main='')
qqline(df_norm$RosterQuality, col = "red")
```



```
levels.roster
```

```
##          (9.77e+06,6.87e+07]          (6.87e+07,1.27e+08]
##   "( 9,770,000, 68,700,000)" "( 68,700,000, 127,000,000)"
##          (1.27e+08,1.85e+08]          (1.85e+08,2.44e+08]
##   "(127,000,000, 185,000,000)" "(185,000,000, 244,000,000)"
##          (2.44e+08,3.02e+08]          (3.02e+08,3.61e+08]
##   "(244,000,000, 302,000,000)" "(302,000,000, 361,000,000)"
##          (3.61e+08,4.19e+08]          (4.19e+08,4.77e+08]
##   "(361,000,000, 419,000,000)" "(419,000,000, 477,000,000)"
##          (4.77e+08,5.36e+08]          (5.36e+08,5.95e+08]
##   "(477,000,000, 536,000,000)" "(536,000,000, 595,000,000)"
```

Same procedure is repeated for Knowledge:

```
# binning of Knowledge feature
df_norm$Knowledge <- cut(df$Knowledge, breaks = 10)
levels.Knowledge <- levels(df_norm$Knowledge)
```

```
# visualize levels of knowledge
levels.Knowledge
```

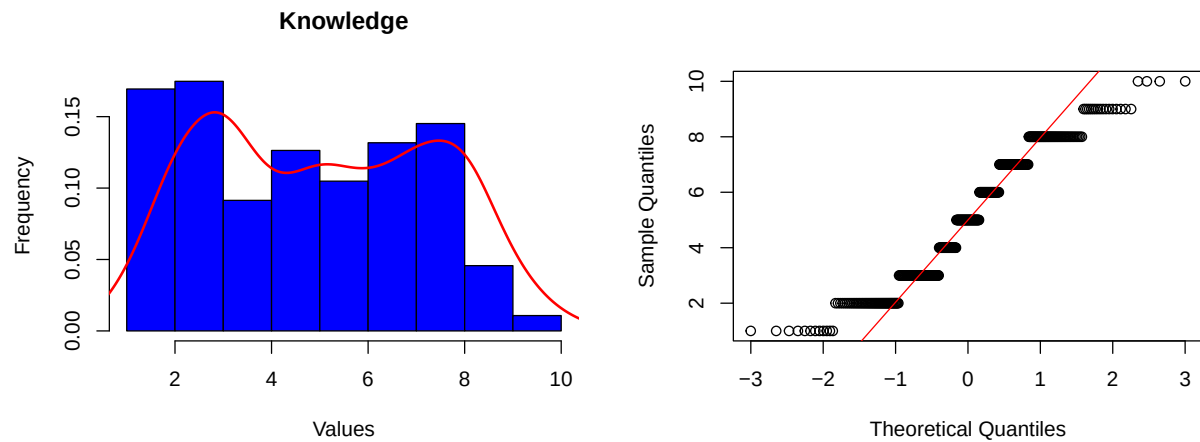
```
## [1] "(23.1,40.6]" "(40.6,57.8]" "(57.8,75.1]" "(75.1,92.4]" "(92.4,110]"
## [6] "(110,127]" "(127,144]" "(144,162]" "(162,179]" "(179,196]"
```

```
# make it numeric
df_norm$Knowledge <- as.numeric(df_norm$Knowledge)

par(mfrow=c(1, 2))
hist(df_norm$Knowledge, main = "Knowledge", xlab = "Values", ylab = "Frequency", col = "blue",
     border = "black",probability = TRUE)

lines(density(df_norm$Knowledge), col = "red", lwd = 2)

qqnorm(df_norm$Knowledge, main='')
qqline(df_norm$Knowledge, col = "red")
```



We originally transformed all the features to be in a $[0,1]$ range, but this transformation caused a decrease in predictive power for the logistic regression model. So we opted to not perform the transformation. Our dataset is now ready to be analyzed!

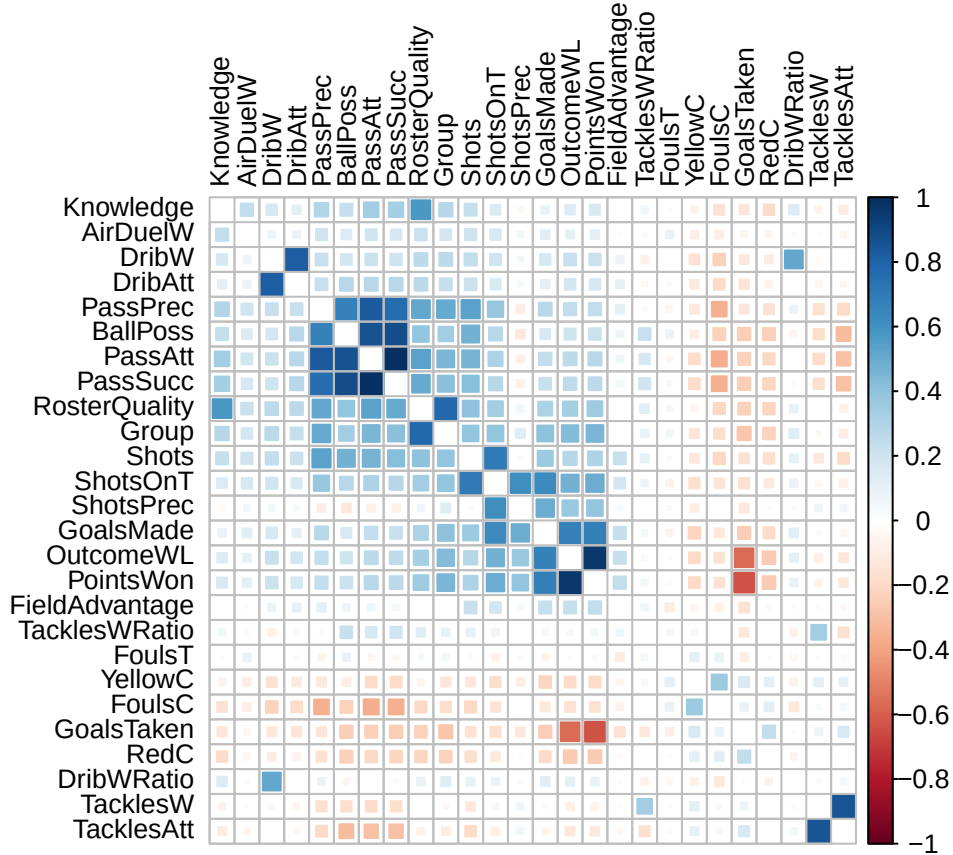
4 Exploratory data analysis:

4.1 Correlation matrix:

A correlation matrix can help us understand the interaction between features:

```
selected_corr <- cor(df_norm)
par(mar=c(1,1,1,1))

corrplot(selected_corr, method = "square", order = "hclust", diag = FALSE, tl.cex = 0.8, number.cex = 0.8,
          tl.col = "black")
```



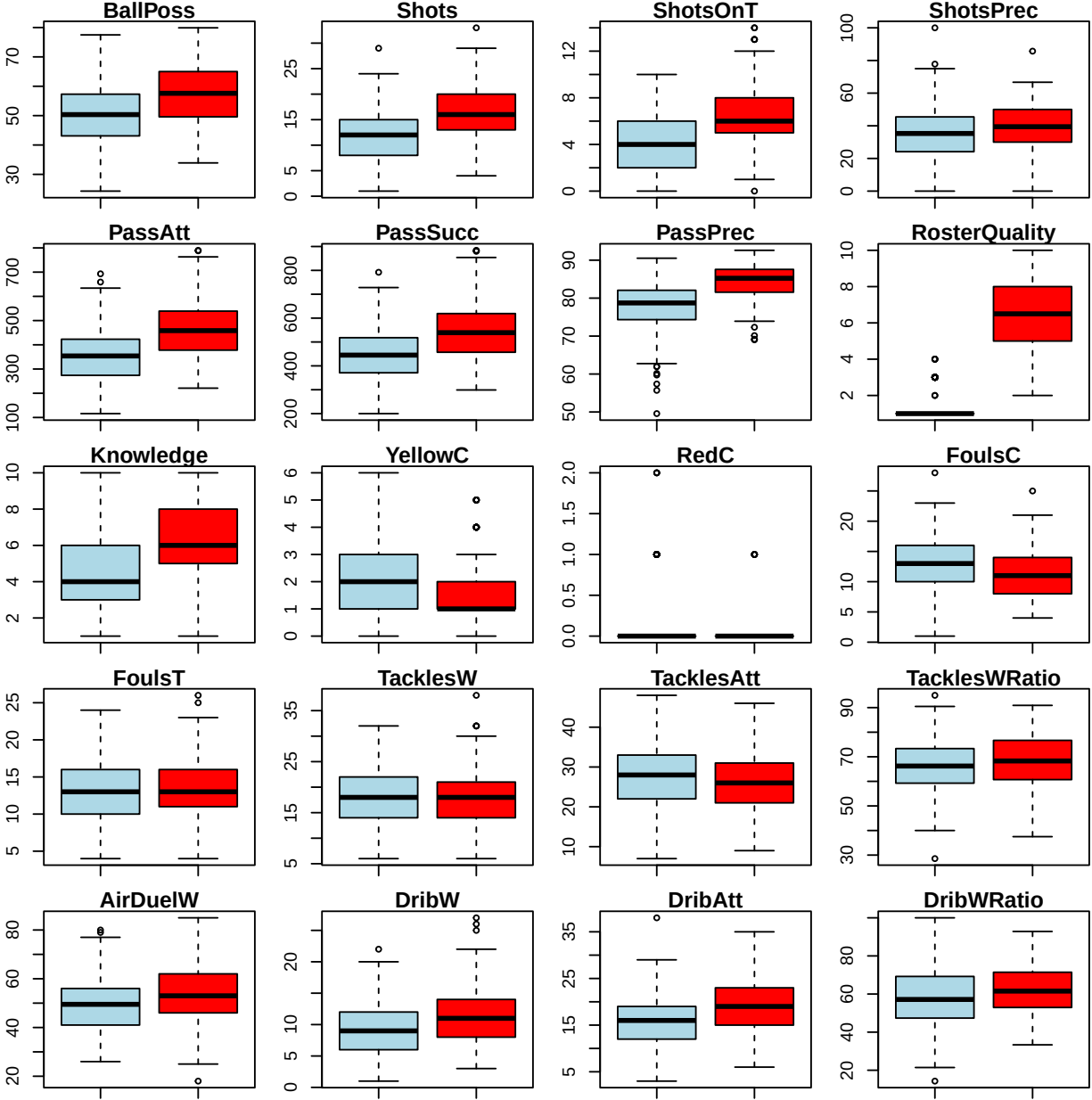
Most results shown by the correlation plot are not surprising: we expect features of the same category to be correlated with on another (eg: Passage attempts with passage successful). Nevertheless, we can derive some interesting insights. By inspecting the “RosterQuality” row we can see that higher quality is strongly correlated with many features that indicate good team performance. We can also look at the “OutcomeWL” row to have a glimpse of which features seem to correlate with winning.

4.2 “League winning” and “control” groups comparison:

We are interested in the differences between our two groups of teams. To compare them we visualize our features on a series of shoulder-to-shoulder boxplots.

```
par(mar=c(1, 2.5, 1, 0.5) + 0.1)
par(mfrow=c(5, 4))

# Generate box plots for each variable
for (col in columns) {
  boxplot(df_norm[[col]] ~ df$Group, df_norm, names = c("", ""), main = paste(col),
    col = c("lightblue", "#FF0000")
  )
}
```



We can already see some patterns emerging. Group 1 teams have overwhelmingly higher Roster Quality as well as considerably higher Knowledge of the league. This explains the skewness previously observed in these features. Teams from group 0 are all squished in the lowest category for Roaster Quality, with just a few outliers falling in higher categories (maximum in the 4th), while teams from group 1 all have very high roster qualities, spanning from second to tenth category with median in the sixth category. This means that most group 0 teams are valued between 10 and 68 million euros, while better teams range from 127 to 595 million euros. This difference in economic terms seems to be translated, as shown by the boxplots, into a difference in performance. Group 1 teams have higher values in all features that, by common sense, indicate good football performance: ball possession, shots statistics, passage, and dribbling statistics. Group 0 teams, on the other hand, seem to require a more physical approach to the game, exhibiting higher numbers of fouls committed. We can confirm which variables are statistically different between the two groups using t-tests:

```

t_test_results <- list()

columns_to_test <- c("BallPoss", "Shots", "ShotsOnT", "ShotsPrec", "PassAtt", "PassSucc",
                    "PassPrec", "YellowC", "RedC", "FoulsC", "FoulsT", "TacklesW", "TacklesAtt",
                    "TacklesWRatio", "AirDuelW", "DribW", "DribAtt", "DribWRatio")

# Loop through each column and perform t-test
for (col in columns_to_test) {
  t_test_result <- t.test(df_norm[[col]] ~ df_norm$Group)
  t_test_results[[col]] <- t_test_result
}

# select p_values to show
p_values <- sapply(t_test_results, function(x) x$p.value)
p_values

```

```

##      BallPoss      Shots      ShotsOnT      ShotsPrec      PassAtt
## 8.595779e-12 2.623802e-15 1.502562e-15 9.152280e-03 1.530309e-20
##      PassSucc      PassPrec      YellowC      RedC      FoulsC
## 6.229597e-17 1.437123e-24 7.647692e-03 1.593800e-05 6.769563e-04
##      FoulsT      TacklesW      TacklesAtt TacklesWRatio      AirDuelW
## 2.515553e-01 5.126928e-01 5.533375e-02 7.339509e-02 3.341756e-04
##      DribW      DribAtt      DribWRatio
## 1.357100e-07 5.693686e-06 9.545196e-03

```

This result tells us that, between all the variables, only fouls taken, tackles won, tackles attempted and tackles win ratio are not significantly different between the two groups. To have a more practical, and numerical, understanding of the differences we compute the difference between the means of the features for the two groups:

```

df_1_norm <- df_norm[df_norm$Group == 1, ]
df_1_norm$Group <- NULL
df_0_norm <- df_norm[df_norm$Group == 0, ]
df_0_norm$Group <- NULL

calc_stats <- function(df) {
  stats <- data.frame(Variable = names(df), Mean = sapply(df, mean))
  return(stats)
}

stats_0 <- calc_stats(df_0_norm)
stats_1 <- calc_stats(df_1_norm)

summary_df <- data.frame(Variable = stats_1$Variable, Mean_1 = stats_1$Mean,
                        Mean_0 = stats_0$Mean)

summary_df$Mean_Difference <- summary_df$Mean_1 - summary_df$Mean_0

summary_df <- summary_df[, c("Variable", "Mean_1", "Mean_0", "Mean_Difference")]

print(summary_df)

```

```

##      Variable      Mean_1      Mean_0 Mean_Difference

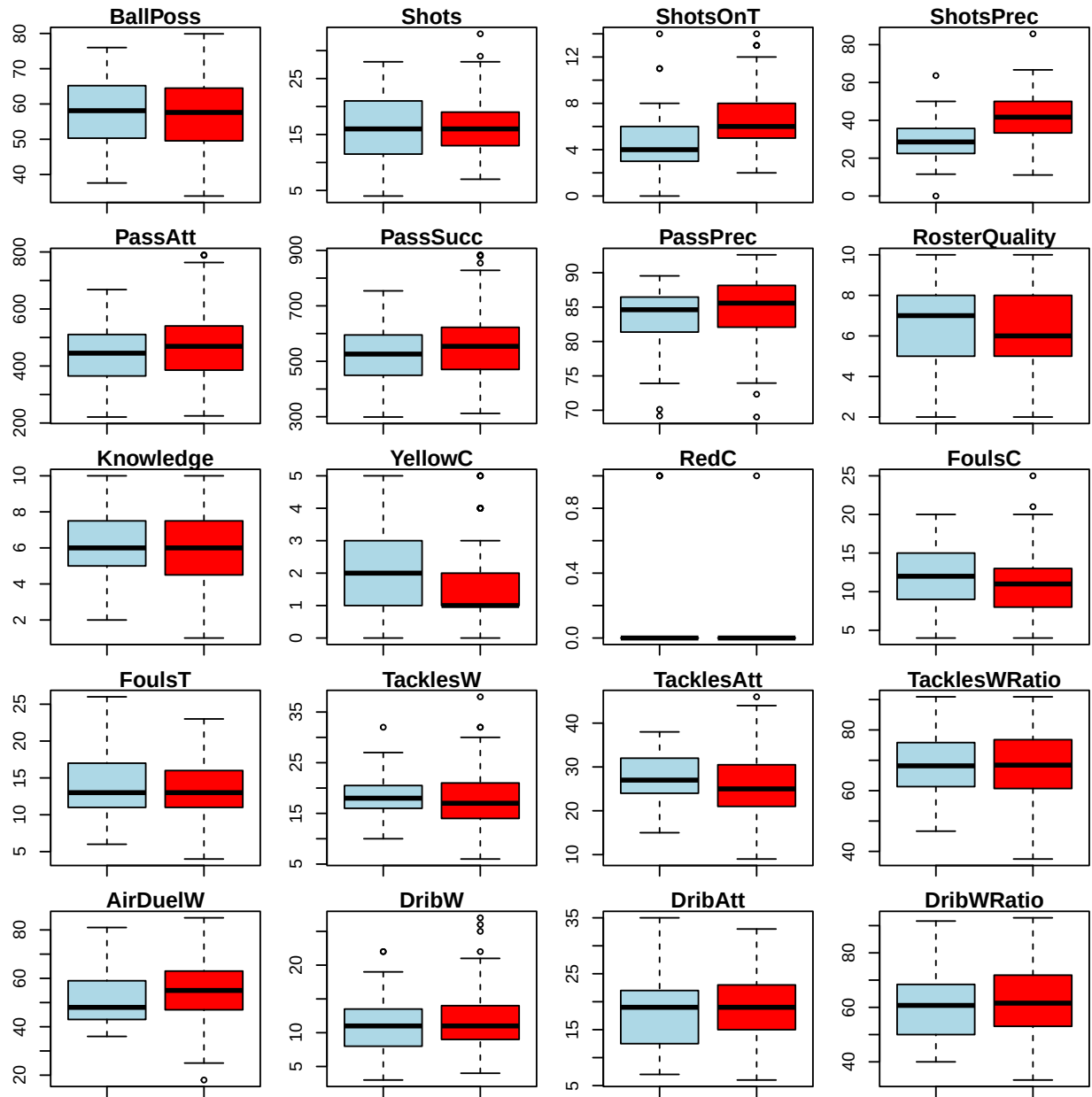
```

## 1	FieldAdvantage	0.50000000	0.50000000	0.00000000
## 2	OutcomeWL	0.76881720	0.33333333	0.4354839
## 3	PointsWon	2.44086022	1.2258065	1.2150538
## 4	GoalsMade	2.49462366	1.2849462	1.2096774
## 5	GoalsTaken	0.82795699	1.5053763	-0.6774194
## 6	BallPoss	57.54462366	50.2048387	7.3397849
## 7	Shots	16.15591398	11.8978495	4.2580645
## 8	ShotsOnT	6.26344086	4.1290323	2.1344086
## 9	ShotsPrec	39.16306452	35.1653226	3.9977419
## 10	PassAtt	466.23118280	352.1290323	114.1021505
## 11	PassSucc	547.90860215	446.8440860	101.0645161
## 12	PassPrec	84.36263441	77.6872581	6.6753763
## 13	RosterQuality	6.05913978	1.4193548	4.6397849
## 14	Knowledge	5.73118280	4.3870968	1.3440860
## 15	YellowC	1.68817204	2.0537634	-0.3655914
## 16	RedC	0.02688172	0.1720430	-0.1451613
## 17	FoulsC	11.38172043	12.8172043	-1.4354839
## 18	FoulsT	13.61290323	13.1397849	0.4731183
## 19	TacklesW	17.86559140	18.2311828	-0.3655914
## 20	TacklesAtt	26.20967742	27.7043011	-1.4946237
## 21	TacklesWRatio	68.44779570	66.4420430	2.0057527
## 22	AirDuelW	53.53763441	49.2258065	4.3118280
## 23	DribW	11.73655914	9.3763441	2.3602151
## 24	DribAtt	18.98387097	16.2150538	2.7688172
## 25	DribWRatio	62.00413978	58.0460753	3.9580645

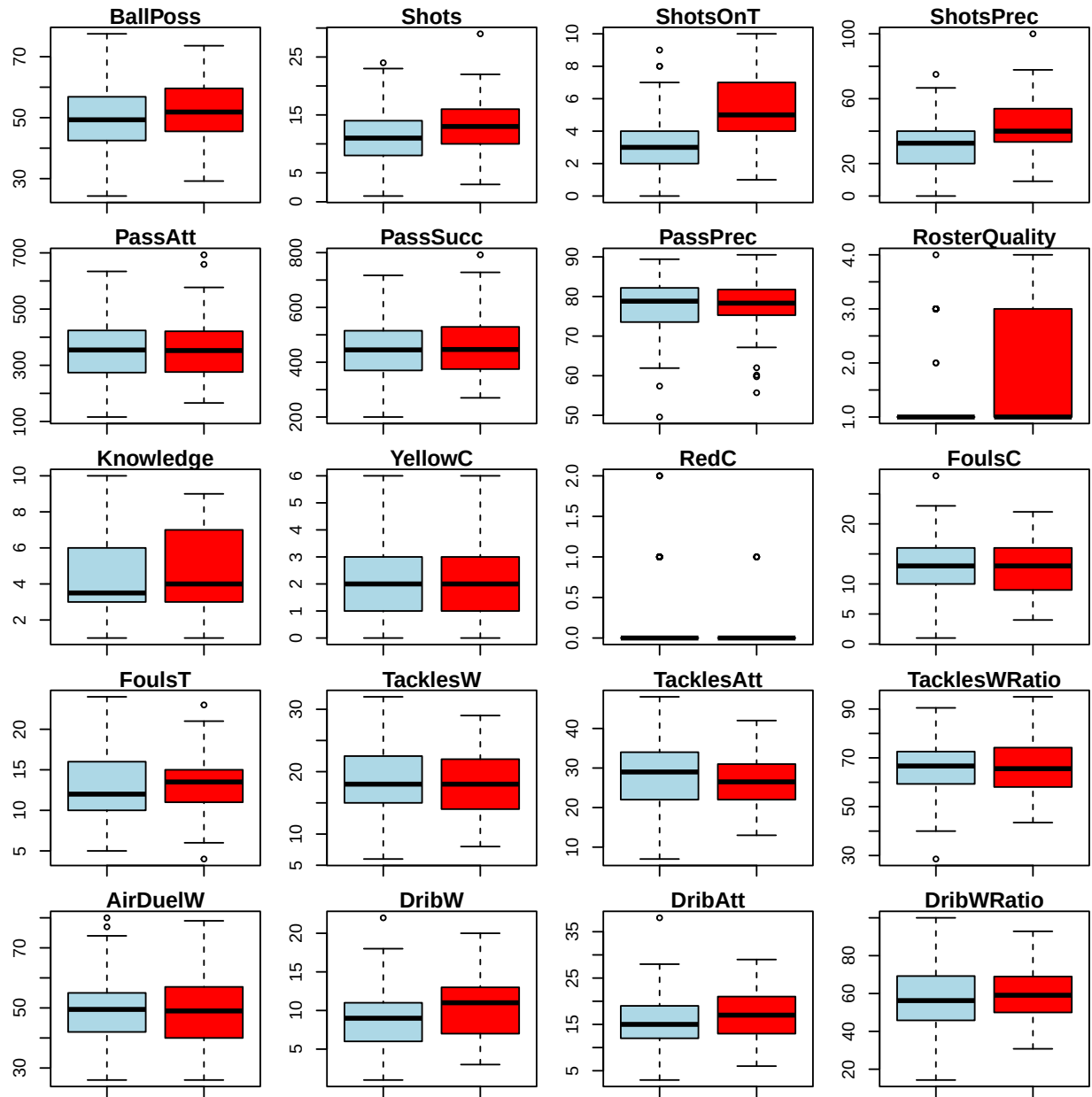
4.3 In-group comparison between won and non-won matches:

We want to check now, inside each group, what are the differences between winning performances and losing ones. We do so by splitting our data and plots, for each group, shoulder-to-shoulder boxplots that compare winning match statistics to losing/tied ones.

```
par(mar=c(1, 2.5, 1, 0.5) + 0.1)
par(mfrow=c(5, 4))
for (col in columns) {
  boxplot(df_1_norm[[col]] ~ df_1_norm$OutcomeWL, df_1_norm, names = c("", ""), main = paste(col),
    col = c("lightblue", "#FF0000"))
}
```



```
for (col in columns) {
  boxplot(df_0_norm[[col]] ~ df_0_norm$OutcomeWL, df_0_norm, names = c("", ""), main = paste(col),
    col = c("lightblue", "#FF0000")
  )
}
```



From these plots, it looks like both good and bad teams maintain similar performances when winning or when losing. The only factor that changes significantly is the number of shots on target. We can confirm this using t-tests as we did previously:

```
t_test_results_1 <- list()

# Loop through each column and perform t-test
for (col in columns_to_test) {
  t_test_result_1 <- t.test(df_1_norm[[col]] ~ df_1_norm$OutcomeWL)
  t_test_results_1[[col]] <- t_test_result_1
}

# select p_values to show
```

```
p_values_1 <- sapply(t_test_results_1, function(x) x$p.value)
p_values_1
```

```
##      BallPoss      Shots      ShotsOnT      ShotsPrec      PassAtt
## 9.957283e-01 6.040894e-01 1.575871e-04 2.198296e-07 1.357957e-01
##      PassSucc      PassPrec      YellowC      RedC      FoulsC
## 1.708747e-01 2.375988e-01 1.409267e-02 6.445835e-02 1.057760e-01
##      FoulsT      TacklesW      TacklesAtt TacklesWRatio      AirDuelW
## 5.433690e-01 1.566060e-01 9.886827e-02 9.861699e-01 2.128026e-01
##      DribW      DribAtt      DribWRatio
## 4.870126e-01 6.988191e-01 5.638300e-01
```

```
t_test_results_0 <- list()

# Loop through each column and perform t-test
for (col in columns_to_test) {
  t_test_result_0 <- t.test(df_0_norm[[col]] ~ df_0_norm$OutcomeWL)
  t_test_results_0[[col]] <- t_test_result_0
}

# select p_values to show
p_values_0 <- sapply(t_test_results_0, function(x) x$p.value)
p_values_0
```

```
##      BallPoss      Shots      ShotsOnT      ShotsPrec      PassAtt
## 1.311312e-01 2.067031e-03 5.625930e-09 5.783842e-06 4.288617e-01
##      PassSucc      PassPrec      YellowC      RedC      FoulsC
## 3.018910e-01 9.558777e-01 1.456977e-01 2.661480e-03 4.776655e-01
##      FoulsT      TacklesW      TacklesAtt TacklesWRatio      AirDuelW
## 7.986241e-01 3.251470e-01 2.312399e-01 9.255146e-01 6.305703e-01
##      DribW      DribAtt      DribWRatio
## 8.775943e-03 3.220160e-02 2.856725e-01
```

For group 1, the only features that behave significantly differently in winning or losing performance are Shots on target and shot precision. Group 0 behaves similarly, with the addition of red cards, dribbles won and dribbles attempted. We now aim to build a prediction model to predict the outcome of a match. Using feature selection algorithms we will try to zero in on the most impactful features.

5 Logistic regression models:

From the correlation matrix, we already know there are some highly correlated features that we can probably exclude from the model. We start with the full model and apply stepwise backward and forward elimination strategies, and see which one performs better.

5.1 Univariate regression:

```
# remove target variables. df_norm.F is our full dataset without target variables
# that we are not interested in at the moment
```

```

df_norm.F <- df_norm
df_norm.F$PointsWon <- NULL
df_norm.F$GoalsMade <- NULL
df_norm.F$GoalsTaken <- NULL
df_norm.F$Group <- NULL

# univariate regression

univariate_results <- data.frame(variable = character(),
                                p_value = numeric(),
                                AIC = numeric(),
                                BIC = numeric())

columns <- c("FieldAdvantage", "BallPoss", "Shots", "ShotsOnT", "ShotsPrec", "PassAtt", "PassSucc",
            "PassPrec", "RosterQuality", "Knowledge", "YellowC", "RedC", "FoulsC", "FoulsT", "TacklesW",
            "TacklesAtt", "TacklesWRatio", "AirDuelW", "DribW", "DribAtt", "DribWRatio")

for (col in columns) {
  glm_u <- glm(OutcomeWL ~ df_norm.F[[col]], data = df_norm.F, family = binomial)
  model_summary <- summary(glm_u)
  pvalue <- model_summary$coefficients[2, "Pr(>|z|)"][1]
  pvalue <- format(pvalue, scientific = FALSE)

  new_row <- data.frame(variable = col,
                        p_value = pvalue,
                        AIC = AIC(glm_u),
                        BIC = BIC(glm_u),
                        stringsAsFactors = FALSE)
  univariate_results <- rbind(univariate_results, new_row)
}

univariate_results

```

##	variable	p_value	AIC	BIC
## 1	FieldAdvantage	0.000009048171	495.5286	503.3664
## 2	BallPoss	0.0001293834	500.3648	508.2026
## 3	Shots	0.0000001575839	484.3724	492.2102
## 4	ShotsOnT	0.0000000000000002481727	418.3818	426.2196
## 5	ShotsPrec	0.000000000008492776	457.1525	464.9903
## 6	PassAtt	0.0000007916248	488.2765	496.1143
## 7	PassSucc	0.00000235177	490.7211	498.5589
## 8	PassPrec	0.000007536621	493.3531	501.1909
## 9	RosterQuality	0.0000000008624432	472.6530	480.4908
## 10	Knowledge	0.004102317	507.3938	515.2316
## 11	YellowC	0.0003153067	502.2454	510.0832
## 12	RedC	0.00002775635	488.6680	496.5058
## 13	FoulsC	0.003131131	506.7840	514.6218
## 14	FoulsT	0.795916	515.7461	523.5839
## 15	TacklesW	0.08908081	512.8918	520.7296
## 16	TacklesAtt	0.01527356	509.8061	517.6439
## 17	TacklesWRatio	0.4650174	515.2783	523.1160
## 18	AirDuelW	0.05044444	511.9267	519.7645
## 19	DribW	0.00002271286	496.1180	503.9558
## 20	DribAtt	0.0004911124	502.9673	510.8051

```
## 21      DribWRatio      0.03249383 511.1559 518.9937
```

5.2 Multivariate regression with feature selection:

```
# -----  
# FEATURE SELECTION  
# we use feature selection techniques to asses which are the most important factors  
# that define a winning performance  
  
results <- data.frame(df = character(), method = character(), error = numeric(), AIC = numeric(),  
                      BIC = numeric())  
  
# full model  
glm_F <- glm(OutcomeWL ~ FieldAdvantage + BallPoss + Shots + ShotsOnT + ShotsPrec + PassAtt + PassSucc  
             + PassPrec + RosterQuality + Knowledge + YellowC + RedC + FoulsC + FoulsT  
             + TacklesW + TacklesAtt + TacklesWRatio + AirDuelW + DribW + DribAtt + DribWRatio,  
             data = df_norm.F, family = binomial)  
  
summary(glm_F)  
  
# backward  
glm_bkw <- step(glm_F, direction = "backward")  
  
summary(glm_bkw)  
  
# forward  
glm_null <- glm(OutcomeWL ~ 1, data = df_norm.F, family = binomial)  
  
glm_fwd <- step(glm_null, scope = list(lower = glm_null, upper = glm_F), direction = "forward")  
  
summary(glm_fwd)  
  
# compute cross validation error with leave-one-out approach  
  
cv_err_full <- cv.glm(df_norm.F, glm_F)  
cv_err_bwd <- cv.glm(df_norm.F, glm_bkw)  
cv_err_fwd <- cv.glm(df_norm.F, glm_fwd)  
  
# Update results dataframe  
  
new_row <- data.frame(df = "Full", method = "Full model", error = cv_err_full$delta[1],  
                     AIC = AIC(glm_F), BIC = BIC(glm_F), stringsAsFactors = FALSE)  
results <- rbind(results, new_row)  
new_row <- data.frame(df = "Full", method = "Backward", error = cv_err_bwd$delta[1],  
                     AIC = AIC(glm_bkw), BIC = BIC(glm_bkw), stringsAsFactors = FALSE)  
results <- rbind(results, new_row)  
new_row <- data.frame(df = "Full", method = "Forward", error = cv_err_fwd$delta[1],  
                     AIC = AIC(glm_fwd), BIC = BIC(glm_fwd), stringsAsFactors = FALSE)  
results <- rbind(results, new_row)  
  
# we can repeat the process with the two separated df to see if the winning conditions  
# change between the groups
```

```

# prepare DF
df_1_norm.F <- df_1_norm
df_1_norm.F$PointsWon <- NULL
df_1_norm.F$GoalsMade <- NULL
df_1_norm.F$GoalsTaken <- NULL
df_1_norm.F$Group <- NULL

df_0_norm.F <- df_0_norm
df_0_norm.F$PointsWon <- NULL
df_0_norm.F$GoalsMade <- NULL
df_0_norm.F$GoalsTaken <- NULL
df_0_norm.F$Group <- NULL

# start with winning group:

glm_F_W <- glm(OutcomeWL ~ FieldAdvantage + BallPoss + Shots + ShotsOnT + ShotsPrec + PassAtt + PassSuc
              + PassPrec + RosterQuality + Knowledge + YellowC + RedC + FoulsC + FoulsT
              + TacklesW + TacklesAtt + TacklesWRatio + AirDuelW + DribW + DribAtt + DribWRatio,
              data = df_1_norm.F,
              family = binomial)
summary(glm_F_W)

# backward
glm_bkw_W <- step(glm_F_W, direction = "backward")

summary(glm_bkw_W)

# forward
glm_null_W <- glm(OutcomeWL ~ 1, data = df_1_norm.F, family = binomial)

glm_fwd_W <- step(glm_null_W, scope = list(lower = glm_null_W, upper = glm_F_W), direction = "forward")

summary(glm_fwd_W)

# compute cross validation error with leave-one-out approach
cv_err_full_W <- cv.glm(df_1_norm.F, glm_F_W)
cv_err_bwd_W <- cv.glm(df_1_norm.F, glm_bkw_W)
cv_err_fwd_W <- cv.glm(df_1_norm.F, glm_fwd_W)

# Update results dataframe
new_row <- data.frame(df = "Group 1", method = "Full model", error = cv_err_full_W$delta[1],
                     AIC = AIC(glm_F_W), BIC = BIC(glm_F_W), stringsAsFactors = FALSE)
results <- rbind(results, new_row)
new_row <- data.frame(df = "Group 1", method = "Backward", error = cv_err_bwd_W$delta[1],
                     AIC = AIC(glm_bkw_W), BIC = BIC(glm_bkw_W), stringsAsFactors = FALSE)
results <- rbind(results, new_row)
new_row <- data.frame(df = "Group 1", method = "Forward", error = cv_err_fwd_W$delta[1],
                     AIC = AIC(glm_fwd_W), BIC = BIC(glm_fwd_W), stringsAsFactors = FALSE)
results <- rbind(results, new_row)

# and finally do the same with control group

glm_F_C <- glm(OutcomeWL ~ FieldAdvantage + BallPoss + Shots + ShotsOnT + ShotsPrec + PassAtt + PassSuc

```

```

+ PassPrec + RosterQuality + Knowledge + YellowC + RedC + FoulsC + FoulsT
+ TacklesW + TacklesAtt + TacklesWRatio + AirDuelW + DribW + DribAtt + DribWRatio,
data = df_0_norm.F,
family = binomial)
summary(glm_F_C)

# backward
glm_bkw_C <- step(glm_F_C, direction = "backward")

summary(glm_bkw_C)

# forward
glm_null_C <- glm(OutcomeWL ~ 1, data = df_0_norm.F, family = binomial)

glm_fwd_C <- step(glm_null_C, scope = list(lower = glm_null_C, upper = glm_F_C), direction = "forward")

summary(glm_fwd_C)

# compute cross validation error with leave-one-out approach
cv_err_full_C <- cv.glm(df_0_norm.F, glm_F_C)
cv_err_bwd_C <- cv.glm(df_0_norm.F, glm_bkw_C)
cv_err_fwd_C <- cv.glm(df_0_norm.F, glm_fwd_C)

# Update results dataframe

new_row <- data.frame(df = "Group 0", method = "Full model", error = cv_err_full_C$delta[1],
                      AIC = AIC(glm_F_C), BIC = BIC(glm_F_C), stringsAsFactors = FALSE)
results <- rbind(results, new_row)
new_row <- data.frame(df = "Group 0", method = "Backward", error = cv_err_bwd_C$delta[1],
                      AIC = AIC(glm_bkw_C), BIC = BIC(glm_bkw_C), stringsAsFactors = FALSE)
results <- rbind(results, new_row)
new_row <- data.frame(df = "Group 0", method = "Forward", error = cv_err_fwd_C$delta[1],
                      AIC = AIC(glm_fwd_C), BIC = BIC(glm_fwd_C), stringsAsFactors = FALSE)
results <- rbind(results, new_row)

# visualize results
results

```

```

##      df      method      error      AIC      BIC
## 1   Full Full model 0.1637666 374.9445 461.1601
## 2   Full  Backward 0.1550963 360.3532 403.4611
## 3   Full   Forward 0.1548526 360.3979 395.6679
## 4 Group 1 Full model 0.1652443 172.4758 243.4422
## 5 Group 1  Backward 0.1309549 151.8776 180.9093
## 6 Group 1   Forward 0.1332943 154.7327 170.8615
## 7 Group 0 Full model 0.1747974 198.3117 269.2781
## 8 Group 0  Backward 0.1585169 182.5513 214.8087
## 9 Group 0   Forward 0.1555086 183.0106 212.0423

```

Accuracy is better with forward model for Full dataframe and Group 0. For Group 1 backward has better accuracy. AIC is better in all 3 cases using backward procedure. BIC is better in all 3 cases using forward procedure. We decide to use model with better BIC, that penalizes the introduction of more features.

5.3 Models diagnostics:

Before obtaining our results we check some diagnostics to see if our models have any problems:

5.3.1 Collinearity:

```
vif(glm_fwd)
```

##	ShotsOnT	RedC	FieldAdvantage	RosterQuality	ShotsPrec
##	1.586464	1.058269	1.093112	1.222265	1.714968
##	TacklesAtt	YellowC	DribW		
##	1.071229	1.024452	1.089405		

```
vif(glm_fwd_W)
```

##	ShotsPrec	RedC	FieldAdvantage	TacklesAtt
##	1.232256	1.075456	1.055605	1.177030

```
vif(glm_fwd_C)
```

##	ShotsOnT	FieldAdvantage	RedC	ShotsPrec	DribW
##	1.711279	1.099057	1.097235	1.690913	1.025350
##	TacklesAtt	PassPrec	PassSucc		
##	1.067792	2.867643	2.503771		

No feature shows VIF higher than 5, so collinearity is not a problem.

5.3.2 ROC and AUC:

```
pred <- predict(glm_fwd, type = "response")
pred_1 <- predict(glm_fwd_W, type = "response")
pred_2 <- predict(glm_fwd_C, type = "response")

roc_obj_F <- roc(df_norm.F$OutcomeWL, pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
roc_obj_1 <- roc(df_1_norm.F$OutcomeWL, pred_1)
```

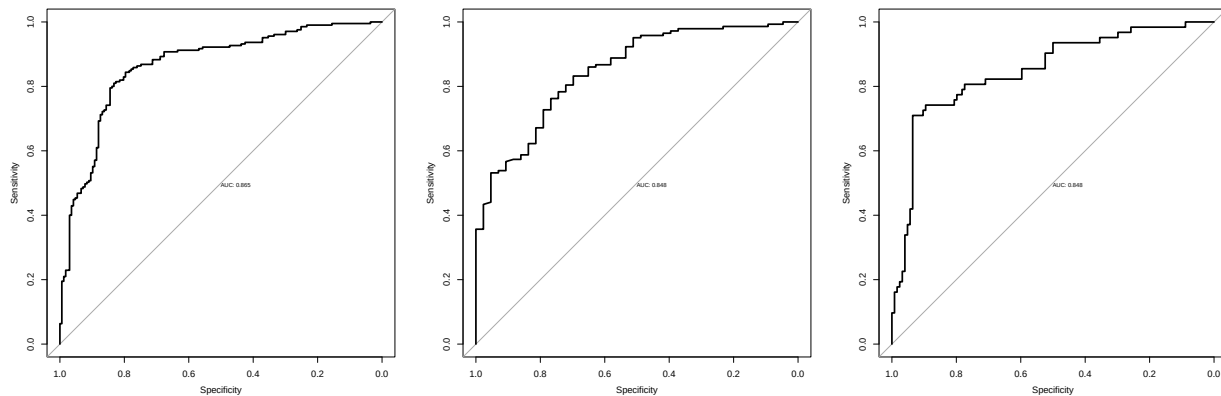
```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
roc_obj_0 <- roc(df_0_norm.F$OutcomeWL, pred_2)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
par(mfrow=c(1, 3))
par(mar = c(4, 4, 2, 2) + 0.1)
plot(roc_obj_F, print.auc=TRUE)
plot(roc_obj_1, print.auc=TRUE)
plot(roc_obj_0, print.auc=TRUE)
```



```
auc(roc_obj_F)
```

```
## Area under the curve: 0.8653
```

```
auc(roc_obj_1)
```

```
## Area under the curve: 0.8479
```

```
auc(roc_obj_0)
```

```
## Area under the curve: 0.8482
```

AUC is >0.8 for all three models, indicating a satisfactory prediction power.

5.4 Feature impact on prediction with odd ratio computation:

By exponentiating the coefficients of the model, we obtain the odd ratios for each selected feature: in this way, we can estimate each feature's impact on winning the match.

```
# full df
coefficients_F <- summary(glm_fwd)$coefficients
odds_ratios_F <- exp(coefficients_F[, "Estimate"])
odds_ratios_F <- format(odds_ratios_F, scientific = FALSE, digits = 4)
odds_ratios_F
```

##	(Intercept)	ShotsOnT	RedC FieldAdvantage	RosterQuality
##	"0.06507"	"1.21298"	"0.11552"	"3.09184"
##	ShotsPrec	TacklesAtt	YellowC	DribW
##	"1.05675"	"0.95180"	"0.84051"	"1.05825"

```
# df 1
coefficients_1 <- summary(glm_fwd_W)$coefficients
odds_ratios_1 <- exp(coefficients_1[, "Estimate"])
odds_ratios_1 <- format(odds_ratios_1, scientific = FALSE, digits = 4)
odds_ratios_1
```

##	(Intercept)	ShotsPrec	RedC FieldAdvantage	TacklesAtt
##	"0.30658"	"1.11877"	"0.02093"	"3.37922"

```
# df 0
coefficients_0 <- summary(glm_fwd_C)$coefficients
odds_ratios_0 <- exp(coefficients_0[, "Estimate"])
odds_ratios_0 <- format(odds_ratios_0, scientific = FALSE, digits = 4)
odds_ratios_0
```

##	(Intercept)	ShotsOnT FieldAdvantage	RedC	ShotsPrec
##	"3.6294"	"1.4890"	"4.9271"	"0.2535"
##	DribW	TacklesAtt	PassPrec	PassSucc
##	"1.0901"	"0.9669"	"0.9050"	"1.0052"

6 Results discussion:

6.1 Overview of Exploratory Data Analysis:

From the Exploratory Data Analysis (EDA), we noticed clear patterns in the data. Teams that won the championship had a significantly higher winning ratio compared to other teams (76% vs 33%). Championship teams had an average roster value between 244 and 477 million euros, whereas the control group teams averaged between 10 and 68 million euros. Although our dataset lacks data from other teams competing for the top League positions, these teams likely have market values comparable to the League winners.

6.2 Team Value and Performance:

We aimed to answer: “What distinguishes a successful team from a mediocre one?” We split the data into two datasets: League-winning teams and a control group, comparing their features. The differences were evident in the “Roster Quality” row of the correlation plot and even clearer in the shoulder-to-shoulder boxplots. Features associated with successful football performance, such as ball possession, shots, and passing statistics, were higher in League-winning teams. Conversely, statistics indicating a more physical game, such as yellow cards, fouls, and tackles, were higher in the control group. To validate these visual results, we conducted pairwise t-tests between the two groups. The tests confirmed that only fouls taken, tackles won, tackles attempted, and tackle-win ratios were not significantly different between the groups. This analysis supports the hypothesis that in football, money buys quality. However, we do not claim that low-valued teams cannot win against high-valued ones or that the highest-valued team will always win the League, but high market value is a prerequisite for being a League contender. Of the league-winning teams we studied, Monaco was the one with lower market value. This is because they were leveraging highly talented players (like Mbappé) not yet fully developed. Mbappé market value after the season skyrocketed to more than 100 million.

6.3 Performance Analysis within Teams:

We then explored the relevant features of a winning performance within the two groups. We hypothesized that winning factors might differ between high and low-value teams. Comparing the features in won versus non-won games through boxplots and t-tests, we found that for winning teams, shots on target and shot precision were significantly different in winning performances. For control group teams, red cards and dribbling statistics were also significant. This suggests that while high and low-quality teams perform similarly in winning and non-winning matches on average, they show more decisiveness in winning performances.

6.4 Logistic Regression Analysis

To gain more insight, we fitted a logistic regression model. Initially, univariate logistic regression with all features was not very useful due to a lack of interaction modeling. Thus, we employed multivariate analysis with backward and forward-step feature selection algorithms. We built three models for the full dataset and the two separate datasets (winners and control). The forward selection model, with a lower BIC score, was chosen for the final results.

6.5 Key Findings

The logistic regression analysis revealed that field advantage is the most critical factor for winning in both groups. For League winners, home advantage increases winning odds by 3.38 times, and for control group teams, by 4.92 times. This difference can be explained by the confidence boost that home advantage provides, particularly to lower-profile teams. Additionally, red cards significantly impact performance, nearly guaranteeing a loss. An increase of 60 million euros in team value results in a 20% increase in winning odds. Tackles attempted were negatively correlated with winning, indicating that reliance on physical play often signifies a struggle to control the match.

7 Actionable insights:

We found different areas where teams could focus to improve their performance, based on the analysis we carried on our data. We will present them, along with some practical ideas that could be reinforced, or implemented by organizations.

7.1 Maximize home Field Advantage

Field advantage is the single most reliable predictor for winning. Both high and low-value teams benefit significantly from playing at home. Coaches and team managers should focus on strategies to maximize this advantage:

- Enhance Home Environment: Improve facilities, increase fan engagement, and create a supportive atmosphere to boost player morale and performance.
- Pre-Match Preparation: Use familiar environments for training sessions before home games to help players feel more comfortable and confident.

7.2 Mental Resilience

Winning performances are linked to better shot precision and decisiveness, indicating the importance of mental aspects. These aspects can also counteract the psychological disadvantage of away matches:

- **Mental Conditioning Programs:** Implement programs that include sports psychology sessions to improve focus, resilience, and pressure handling.
- **Stress Management Techniques:** Teach players techniques like mindfulness, visualization, and breathing exercises to maintain composure during crucial moments.

7.3 Reduce Physical Play in Low-Value Teams

For lower-value teams, excessive physical play, indicated by higher fouls and tackles, correlates negatively with winning:

- **Discipline and Control:** Train players to avoid unnecessary fouls and maintain discipline on the field to reduce red card incidents.
- **Tactical Training:** Focus on improving ball control and possession skills to minimize the need for aggressive physical play.

7.4 Optimize Key Performance Metrics

Both high and low-value teams should focus on improving specific metrics identified as critical for winning:

- **Shots on Target and Precision:** Develop specialized training sessions to enhance shooting accuracy and conversion rates.
- **Passing and Ball Possession:** Invest in drills that improve passing accuracy and maintain possession, as these are critical for high-value teams.

7.5 Roaster quality

Investing in high-quality players significantly boosts team performance, but strategic investments are essential:

- **Targeted Recruitment:** Focus on recruiting players who excel in key areas such as shooting precision, passing, and mental resilience.
- **Youth Development:** For teams with less financial capabilities, developing a robust youth academy system to nurture talent internally is especially important, as seen in Monaco's example.

7.6 Leverage Data Analytics

Utilize data analytics to continually monitor and improve team performance:

- **Performance Tracking:** Implement advanced data tracking systems to analyze player and team performance metrics in real-time.
- **Data-Driven Decisions:** Couple the coach's and staff's intuition with solid data analysis, to improve decisions about training, strategy, and player selection.
- **Competition study:** Use data analysis to study other teams' performances and discover potential weak spots in their composition or tactics.